

Relatório de Criptografia 2

- Onde está Stuart Little?

Sabe-se que Stuart Little está no ponto $P + Q$, sendo $P = (2, 7)$ e $Q = (2, 36)$. Pela aritmética modular, é possível escrever o ponto Q em sua forma equivalente modular negativa, na qual $Q = (2, 36) = (2, -7)$, já que $36 - 43 = -7$.

Dessa forma, observa-se que o ponto $Q = (2, -7)$ é o oposto modular de $P = (2, 7)$. Quando é feita uma soma de dois pontos opostos modulares (pertencem à reta vertical ao eixo X), sua soma é definida como ponto no infinito, também denotado pelo símbolo "O". Assim, a flag entregue ao chall foi *CTF-BR{O}*.

- Tem na PrimeVideo

Como mencionado na descrição do chall, a criptografia empregada neste problema é o ECDSA, utilizado para assinar digitalmente mensagens. Pelo fato de Alex ter utilizado o mesmo valor de k para gerar duas assinaturas distintas, Rosie consegue realizar um ataque ao ECDSA que possibilita a recuperação da chave privada usada na comunicação.

Como descrito no PDF da GET de Criptografia 2, página 13, os valores de k e d_A podem ser obtidos da seguinte forma algébrica:

Dessa forma, k pode ser facilmente obtido com o cálculo abaixo

$$k \equiv (z_1 - z_2) \cdot (s_1 - s_2)^{-1} \pmod{|E|}$$

Com k em mãos, é possível colocar tudo em função de d_A a partir da congruência

$$s_1 \equiv ((z_1 + rd_A) \cdot k^{-1}) \pmod{|E|}$$

Novamente, multiplicamos k de ambos os lados

$$(z_1 + rd_A) \equiv s_1 \cdot k \pmod{|E|}$$

$$rd_A \equiv (s_1 \cdot k - z_1) \pmod{|E|}$$

Portanto, a chave privada é a solução de

$$d_A \equiv (s_1 \cdot k - z_1) \cdot r^{-1} \pmod{|E|}$$

Em código, as expressões de k e d_A ficam da seguinte maneira, no qual o pow é utilizado para fazer exponenciação modular e o valor do módulo é denotado por n , ao invés do $|E|$ utilizado no PDF.

```
1 import hashlib
2 from Crypto.Util.number import long_to_bytes
3 def sha1(mensagem):
4     m = hashlib.sha1()
5     m.update(mensagem.encode('utf-8'))
6     return m.hexdigest()
7
8 # módulo n (equivalente ao |E| descrito nas anotações da GET)
9 n = 65656373815618361419628414421875556221871775444997987269985123636136484225495156253358957218581173573426962
10
11 # Assinatura Digital 1 (Perceba que r é igual para as duas assinaturas)
12 r=9560785699842885696286466742723019779774182690698735206651761870272674103420578870624360822163833643921876618
13 s1=221531213777841923648016978735137443328287049470572243281503729853749410439027043274206080010606200779066290
14 mensagem1 = "There's the pangs of time, there's the undiscover'd country from whose bourn no traveller returns,"
15
16 # Assinatura Digital 2
17 r=9560785699842885696286466742723019779774182690698735206651761870272674103420578870624360822163833643921876618
18 s2=550675907027773435340732943826706503566101983858018736543389337102898046242499035786904736386910311774088874
19 mensagem2 = "What down at the name of Sanders. (What down at the forest all by himself. It went like that, just
20
21 z1,z2 = int(sha1(mensagem1), 16), int(sha1(mensagem2), 16)
22 # Desafio Começa aqui :
23
24 k = ((z1 - z2) * pow(s1 - s2, -1, n)) % n
25
26 dA = ((s1 * k - z1) * pow(r, -1, n)) % n
27
28 # Desafio acaba aqui! =)
29 print(long_to_bytes(dA))
```

b"CTF-BR{oi,R0si3_s4bia_qu3_ir14_c0ns3gu1r!_Qu3_t4L_ass1st1rm0s_'Simplesmente Acontece'_m41s_t4rde?}"

- ElGaton

Esse chall se baseia na Variante de Menezes e Vanstone para o ElGamal, que usa Criptografia de Curvas Elípticas. O primeiro passo para a decifração da mensagem é encontrar o valor de T a partir da função " $encontrar_nP(k, P, a, p)$ " disponível no código, definindo seus parâmetros através das informações que temos, tal como as variáveis Q_{sam} , n_{frodo} e P .

Analisando a função " $encontrar_nP(k, P, a, p)$ ", verifica-se que k é um inteiro que representa o fator multiplicativo na curva elíptica ($T = kP$), P é o ponto inicial na curva representado em coordenadas (X, Y) , a é um dos coeficientes da equação da curva elíptica e p é o módulo utilizado para definir a curva. Dessa forma, por assimilação direta, sabe-se que os parâmetros a e p são iguais às suas respectivas variáveis já definidas no código, de mesmo nome. Por outro lado, podemos supor que n_{frodo} é a chave privada k que este usaria para codificar seu ponto enquanto Q_{sam} é o ponto pertencente a Sam após aplicado sua própria chave privada junto do ponto escolhido na comunicação, ainda mais que está em coordenadas (X, Y) . Sendo assim, a função fica no formato $encontrar_nP(n_{frodo}, Q_{sam}, a, p)$ e serve para encontrar T , o ponto usado para encriptar a mensagem.

A partir de T , podemos utilizar as seguintes fórmulas apresentadas no PDF para descobrir o X e o Y decriptados, fazendo um pouco de manipulação algébrica para isolar m_1 e m_2 . Vale notar que x_t^{-1} e y_t^{-1} são respectivamente a exponenciação modular da primeira e segunda posição do output da função $encontrar_nP(n_{frodo}, Q_{sam}, a, p)$, ou seja, $T[0]$ e $T[1]$.

6.2 Encriptando e Decriptando a Mensagem

Como Alice e Bob já trocaram suas chaves na etapa do Diffie-Hellman e ambos já possuem T , então Alice encripta a mensagem m em uma simples passo:

$$x_S \equiv x_T \cdot m_1 \pmod{n}$$

$$y_S \equiv y_T \cdot m_2 \pmod{n}$$

Em seguida, ela envia a mensagem encriptada (x_S, y_S) para Bob, que por sua vez, pode decriptar a mensagem ao encontrar o inverso multiplicativo de sua chave $T = (x_T, y_T)$ em $\mathbb{Z}_n$.

$$x_T^{-1} \cdot x_S \equiv x_T^{-1} \cdot x_T \cdot m_1 \equiv m_1 \pmod{n}$$

$$y_T^{-1} \cdot y_S \equiv y_T^{-1} \cdot y_T \cdot m_2 \equiv m_2 \pmod{n}$$

Perceba que para isso precisamos $\text{mdc}(x_T, n) = \text{mdc}(y_T, n) = 1$, para existirem x_T^{-1} e $y_T^{-1} \in \mathbb{Z}_n$. Quando n é primo e $x_T, y_T < n$ garantimos essa condição.

```

32 # Chave Pública
33 # y^2 = x^3 + ax + b (mod p)
34 a = 15347898055371580590890576721314318823207531963035637503096292
35 b = 7444386449934505970367865204569124728350661870959593404279615
36
37 P = (1619092589586542907492569170434842128165755668543894279235270,
38      | 3436949547626524920645513316569700140535482973634182925459687)
39
40 p = 17676318486848893030961583018778670610489016512983351739677143
41
42 # Mensagem Encriptada
43 S = (33472336424414961000482608007001766504544326753246721049147, 12254736703215694149846183808068667208423268)
44
45 # Diffie-Hellmann
46
47 Q_sam= (246061765250195212969506118903120091083185433952208069588975, 934375436288975056976695404137341435878768)
48
49 n_frodo = 233336110442550297617299
50
51 # Altere SOMENTE esse trecho abaixo para obter a mensagem decriptada
52 # Um exemplo seria: 3P = encontra_nP(3,P,a,p)
53 T = encontra_nP(n_frodo, Q_sam, a, p)
54
55 dec_x = (S[0] * pow(T[0], -1, p)) % p # xs * xt^-1 mod p
56 dec_y = (S[1] * pow(T[1], -1, p)) % p # ys * yt^-1 mod p
57 # Até aqui
58
59 # Essa função retorna a flag, basta alterar o trecho acima e você terá a resposta =)
60 print(long_to_bytes(int(str(dec_x)+" "+str(dec_y))))

```

b'CTF-BR{miau_m30w_me0w_mi4u_m3ow_meow_m1au_m14u}'

